

Claude Code Token Optimization Toolkit

Cut your Claude Code spend 60–90% with 4 free, open-source tools — benchmarked on real sessions, not marketing copy.

47%

Output token reduction
measured on real sessions

4

Free tools covered
with install steps

30m

Full stack setup
start to finish

extraeearn.com

github.com/extraeearn/claude-code-token-toolkit

2026 · Updated quarterly

\$29

One-time · Instant download

Claude Code Token Optimization Toolkit

Executive Summary — Claude Code bills on four token buckets: input, output, cache creates, and cache reads. Most users focus on raw input, but output tokens (5× more expensive) and cache creates are the real cost drivers. This guide covers four free tools that together produce **85–92% total token reduction** on a typical build session. Real benchmark: Caveman alone cut output tokens **47%** in one day. Install time: 30 minutes.

Table of Contents

1. [Why Claude Code Burns Tokens](#)
 2. [Tool #1: Caveman + CLAUDE.md Optimization](#)
 3. [Tool #2: context-mode + Codebase Memory MCP](#)
 4. [Stack These Together: Full Setup in 30 Minutes](#)
-

Why Claude Code Burns Tokens

Claude Code is priced on tokens — the chunks of text and data that flow in and out of the model on every exchange. The more tokens, the higher the bill. That part most people understand.

What most people don't understand is *where the tokens actually go*.

The Four Buckets

Every Claude Code session draws from four token buckets, billed at very different rates:

BUCKET	WHAT GOES HERE	COST (SONNET 4.6)
Input	Your messages, file contents, command output	\$3.00 / 1M tokens
Output	Claude's responses	\$15.00 / 1M tokens
Cache Create	Context stored in cache for reuse	\$3.75 / 1M tokens
Cache Read	Re-reading that cached context	\$0.30 / 1M tokens

Output tokens are the most expensive — 5× the input rate. But the real drain in a long Claude Code session isn't any single message. It's the *accumulating context* that gets re-loaded on every exchange.

How Context Accumulates

Every time you send Claude Code a message, the model receives:

1. Your full CLAUDE.md (loaded fresh, every single time)
2. Any files it read earlier in the session
3. Every tool call result — including complete output of every bash command

4. Its own previous responses

By hour two of a build session, Claude Code isn't just processing your latest question. It's re-reading 30,000 tokens of history, 50 lines of CLAUDE.md, and the full output of every `npm install` it ran along the way.

What Prompt Caching Does (And Doesn't Do)

Prompt caching stores repeated context blocks and re-reads them at \$0.30/MTok instead of \$3.00/MTok — a 10× reduction. This is why a typical optimized setup shows ~96% of tokens as cheap cache reads. But two things caching doesn't help with:

1. **Output tokens** — every response is billed at \$15/MTok regardless
2. **Cache creates** — the first load of any large context block is billed at \$3.75/MTok

The Three Levers

Lever 1: Shrink what loads every message. Your CLAUDE.md loads on every exchange — 852 tokens costs 2.8× more per message than 300 tokens.

Lever 2: Stop output tokens inflating. Caveman rewrites Claude's output style to be terse without losing technical accuracy. Output tokens drop 40–65% on prose-heavy responses.

Lever 3: Stop large outputs hitting context raw. context-mode intercepts big outputs and returns summaries. Codebase Memory MCP replaces file reads with graph queries.

Tool #1: Caveman + CLAUDE.md Optimization

These two tools target your biggest cost levers and cost nothing to run.

Caveman

What it does: Rewrites Claude’s own responses to be maximally terse. Code output is untouched. Only narrative text compresses.

GitHub: <https://github.com/JuliusBrussee/caveman> • **Cost:** Free •

Install: ~2 min

Real Before/After

Measured across two consecutive Claude Code days (same project, Sonnet 4.6):

DAY	CAVEMAN	OUTPUT TOKENS	COST
Jun 9 (baseline)	OFF	52,759	\$4.23
Jun 10 (active)	ON	27,794	\$3.96

Output tokens down 47%. Jun 9 was a heavier session, so this isn’t a clean A/B — the honest community range is 40–65% across typical sessions.

Install

```
curl -fsSL https://raw.githubusercontent.com/JuliusBrussee/caveman
```

Wires Claude Code hooks, stats tracker, and status line badge automatically.

Activate

```
/caveman
```

Run at session start. Three intensity levels: `lite`, `full` (default), `ultra`.

Biggest limitation

Caveman only affects output tokens (~0.67% of total volume in a well-optimized setup). On a \$9/week spend, 47% output reduction saves ~\$1.50–\$2.50/week. Not transformative alone — but it stacks with zero cost or config.

CLAUDE.md Optimization

What it does: Strips CLAUDE.md to the minimum Claude actually needs. Every excess token loads into every message of every session.

Cost: Free. No install. One hour of editing.

Real Before/After

METRIC	BEFORE	TARGET	REDUCTION
Est. tokens	~852	~300	65%

Audit your CLAUDE.md

```
bash demos/benchmark_claudemd.sh /path/to/your/CLAUDE.md
```

What to strip

Three categories of dead weight in most CLAUDE.md files:

1. **Things Claude reads from code** — file structure, framework choices, obvious conventions
2. **Process docs** — workflow descriptions, PR procedures
3. **Soft preferences** — anything you wouldn't block a merge over

Target format (under 300 tokens):

```
# [Project Name]

## Rules
- [hard constraint – one line each]

## Stack
- [key tech only]

## Never
- [explicit prohibitions only]
```

Combined impact

TOOL	OUTPUT TOKENS	CACHE CREATES	EST. WEEKLY SAVINGS
Caveman	-47%	—	~\$1-2/week
CLAUDE.md	—	-65% on CLAUDE.md portion	~\$1-2/week
Combined			~\$2-4/week on \$9 baseline

Tool #2: context-mode + Codebase Memory MCP

These two tools target cache creates from large file reads and command output.

context-mode

What it does: Intercepts large tool outputs and returns a structured summary instead of raw content.

GitHub: <https://github.com/sgaabdu4/claude-code-tips> • **Cost:** Free • **Install:** ~5 min

Real Before/After (community benchmark, 15 sessions)

SCENARIO	WITHOUT	WITH CONTEXT-MODE	REDUCTION
Large file read (500+ lines)	~8,000 tokens	~160 tokens	98%
<code>npm install</code> output	~3,200 tokens	~200 tokens	94%
Test suite output (50 tests)	~5,500 tokens	~300 tokens	95%

Install

```
git clone https://github.com/sgaabdu4/claude-code-tips ~/.claude/
```

Set line threshold to 200. Any output above that gets summarized.

Biggest limitation

Summaries can miss edge-case detail. Toggle off for debugging sessions, on for building:


```
context-mode off    # debugging
context-mode on     # building
```

Codebase Memory MCP (CBM)

What it does: Persistent knowledge graph of your codebase. Claude queries the graph instead of reading files.

GitHub: <https://github.com/sgaabdu4/claude-code-tips> • **Cost:** Free • **Install:** ~10 min + index

Real Before/After (community benchmark)

METRIC	WITHOUT CBM	WITH CBM	REDUCTION
"Where is X defined?"	~12,000 tokens	~400 tokens	97%
Session 2 cold start	baseline	~60–80% of file-read input	60–80%

First session pays indexing cost. Every session after reads from the graph.

Install

```
claude mcp add codebase-memory-mcp
```

Index per project (run once):

```
/cbm:index
```

After major refactors: `/cbm:reindex`.

Stack These Together: Full Setup in 30 Minutes

Install in this exact order — measurement accuracy depends on adding one tool at a time.

Step 1: Baseline (~5 min)

```
npm install -g ccusage && ccusage report
```

Note your weekly output tokens and cost. Everything measures against this.

Step 2: Strip CLAUDE.md (~20 min, one-time)

```
bash demos/benchmark_claudemd.sh /path/to/CLAUDE.md
```

Target: under 300 tokens. **Do this first** — permanent, zero-cost, loads every message.

Step 3: Install Caveman (~2 min)

```
curl -fsSL https://raw.githubusercontent.com/JuliusBrussee/caveman
```

Activate: `/caveman` — Check delta after one day: `ccusage report --compare`

Step 4: Install context-mode (~5 min)

```
git clone https://github.com/sgaabdu4/claude-code-tips ~/.claude/
```

Set line threshold to 200. Test with a large file read.

Step 5: Install CBM (~10 min)

```
claude mcp add codebase-memory-mcp
```

Index first project: `/cbm:index`

Verify

After one full week: `ccusage report --compare`

Expected vs. baseline: output -40-65%, cache creates -50-80%, total cost -40-70%.

Community benchmark (15 sessions): 85-92% total token reduction vs. vanilla Claude Code.

Quick Reference

TOOL	INSTALL	WHAT IT SHRINKS	WHEN IT MATTERS
CLAUDE.md	Manual edit	Cache creates (every message)	Always
Caveman	2 min script	Output tokens	Every session
context-mode	5 min clone	Cache creates (large outputs)	File-heavy sessions
CBM	10 min + index	Cache creates (file reads)	Multi-session, large repos

Troubleshooting

Caveman not activating: Run `/caveman` explicitly. Check `~/.claude/hooks/` for installed hooks.

context-mode over-summarizing: Lower line threshold or `context-mode off` for debugging.

CBM index stale: Run `/cbm:reindex` after major refactors.

ccusage showing no change: Add tools one at a time and compare after each. One tool may not be wired correctly.

Claude Code Token Optimization Toolkit • [extraearners.com](#) • [github.com/extraearners/claude-code-token-toolkit](#) • Updated quarterly